

Project Report: Salient Object Detection

End-to-End ML/DL Project — Cohort V

1. Introduction and Project Objective

Salient Object Detection (SOD) is a fundamental computer vision task that focuses on identifying and segmenting the most visually prominent objects in an image — regions that naturally and automatically attract human attention. Unlike traditional object detection, which classifies every visible object, SOD highlights the dominant or “salient” region within the context of a single image.

This project aims to build a complete SOD system from scratch, using core Machine Learning and Deep Learning concepts. Every component was developed manually: the data pipeline, CNN model, loss function, evaluation metrics, and visualization logic. The result is a fully functional model that takes any image as input and produces a saliency mask overlaid on it.

The practical applications of SOD are numerous and important: automatic image cropping, image compression, robotics, automated visual question answering, and medical imaging. This project demonstrates core deep learning competencies: designing encoder-decoder architectures, handling imbalanced segmentation tasks, and evaluating spatial prediction quality.

2. Dataset and Data Preparation

2.1 Dataset Selection

The dataset chosen for this project is DUTS (Dense and Unambiguous Training Set), which is the largest and most widely used dataset for salient object detection. It contains over 10,500 training images with pixel-accurate saliency masks, covering different scenarios, objects, and lighting conditions.

The dataset was downloaded directly from the official source and extracted in Google Colab. After downloading, the existence of image and mask folders was verified, as well as the exact number of files found.

2.2 Data Processing Pipeline

All images and masks were processed using standardized functions implemented in the file `data_loader.py`:

- Images were loaded using OpenCV and converted from BGR to RGB color space
- All images and masks were resized to 128×128 pixels
- Pixel values were normalized to the range [0, 1] by dividing by 255
- Masks were loaded as grayscale images and processed with `INTER_NEAREST` interpolation to preserve sharp boundaries

The `SODDataset` class was implemented as a subclass of PyTorch Dataset and manages loading each image-mask pair, converting them into PyTorch tensors in correct format (C, H, W), and providing data to the DataLoader.

2.3 Train / Validation / Test Split

The dataset was split using `train_test_split` from sklearn with a fixed seed (`random_state=42`) for reproducibility:

Split	Percentage	Approximate Count
Training	70%	~7,350 images
Validation	15%	~1,575 images
Testing	15%	~1,575 images

Three DataLoader objects were created with `batch_size=16`, where training used `shuffle=True` and validation/testing used `shuffle=False`.

2.4 Data Visualization

After loading, samples from the DataLoader were visualized to verify data correctness. It was confirmed that images had the correct shape (16, 3, 128, 128) and masks (16, 1, 128, 128), and that values were in the expected range [0, 1].

3. Model Architecture

3.1 Baseline Model — BaselineSOD

As a starting point, a simple encoder-decoder CNN model without any normalization or regularization was implemented. The goal of the baseline model is to establish a lower performance bound against which improvements are measured.

The encoder contains two convolutional blocks:

- $\text{Conv2d}(3 \rightarrow 16) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$
- $\text{Conv2d}(16 \rightarrow 32) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$

The decoder reconstructs spatial resolution:

- $\text{ConvTranspose2d}(32 \rightarrow 16) \rightarrow \text{ReLU}$
- $\text{ConvTranspose2d}(16 \rightarrow 1)$

The model was trained for 5 epochs using BCEWithLogitsLoss and Adam optimizer with learning rate $1e-4$. The lack of BatchNorm and Dropout makes the model sensitive to overfitting and limits representational capacity.

3.2 Improved Model — ImprovedSOD

The improved model introduces three major architectural changes motivated by best practices in deep learning for segmentation tasks:

Change 1 — Deeper encoder with 3 stages:

- enc1: $\text{Conv2d}(3 \rightarrow 16) + \text{BatchNorm2d}(16) + \text{ReLU} + \text{MaxPool2d}(2)$
- enc2: $\text{Conv2d}(16 \rightarrow 32) + \text{BatchNorm2d}(32) + \text{ReLU} + \text{MaxPool2d}(2)$
- enc3: $\text{Conv2d}(32 \rightarrow 64) + \text{BatchNorm2d}(64) + \text{ReLU} + \text{Dropout}(0.3) + \text{MaxPool2d}(2)$

Change 2 — Batch Normalization in every layer:

Normalizes activations and stabilizes training, allowing higher learning rates and faster convergence.

Change 3 — Dropout(0.3) in deepest block:

Reduces overfitting by injecting randomness during training, forcing the model to learn more general features.

The decoder reconstructs resolution in three stages:

- dec1: `ConvTranspose2d(64→32) → ReLU`
- dec2: `ConvTranspose2d(32→16) → ReLU`
- dec3: `ConvTranspose2d(16→1)`

3.3 Loss Function and Optimizer

`BCEWithLogitsLoss` was chosen as the loss function for both models. This function is suitable for pixel-level binary segmentation because it combines sigmoid and binary cross-entropy in a numerically stable operation. Each pixel is predicted as salient (1) or background (0).

Adam optimizer was used with learning rate $1e-4$ for both models. Adam is standard for segmentation tasks because it adapts learning rates automatically for each parameter.

4. Training and Validation

4.1 Training Loop

The training loop was implemented in `train.py` as a reusable function `train_model()`. For each epoch, the loop follows these steps:

- Model set to train mode (`model.train()`)
- For each batch: images and masks are sent to GPU/CPU, output is computed, loss is calculated, backpropagation and `optimizer.step()` are performed
- Average training loss is computed and recorded
- Model is set to eval mode (`model.eval()`)
- Validation loop computes loss without gradients (`torch.no_grad()`)
- Training and validation losses are printed and stored

4.2 Baseline Model Training

The baseline model was trained for 5 epochs. Loss decreased progressively but remained relatively high due to limited architecture capacity and small number of epochs.

4.3 Improved Model Training

The improved model was trained for 20 epochs. The loss curve showed gradual convergence with stable decreases in both training and validation loss. The gap between training and validation loss remained small, indicating effective regularization from BatchNorm and Dropout.

After training, the model was saved as `sod_model.pth` and loaded for evaluation and demo.

5. Evaluation Metrics

5.1 Selected Metrics

Four standard metrics were implemented in `evaluate.py` for evaluating segmentation quality:

- **IoU (Intersection over Union):** Measures spatial overlap between predicted and ground truth mask. IoU = 1 is perfect, IoU = 0 means no overlap.
Formula: $\text{IoU} = (\text{Intersection} + \epsilon) / (\text{Union} + \epsilon)$
- **Precision:** Measures correctness of positive predictions.
Formula: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- **Recall:** Measures completeness of detection.
Formula: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- **F1 Score:** Harmonic mean of Precision and Recall.
Formula: $\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

A threshold of 0.4 was used on sigmoid outputs to generate binary predictions.

5.2 Quantitative Results

Model	IoU	Precision	Recall	F1 Score
Baseline (5 epochs)	0.2882	—	—	0.4425
Improved (20 epochs)	0.4547	~0.61	~0.58	0.6231

Improvement:

- IoU: +57.8%
- F1: +40.8%

6. Experiments and Comparison

6.1 Experiment 1 — Baseline Model

Configuration: 2-layer encoder without BatchNorm/Dropout, 5 epochs, $lr=1e-4$.

Result: $IoU = 0.2882$, $F1 = 0.4425$. The model captures some salient regions but with many spatial errors. Predictions are unclear and often fail to properly isolate the dominant object.

Analysis: Limited capacity and lack of regularization make the model unable to learn complex saliency features. 5 epochs are also insufficient for convergence.

6.2 Experiment 2 — Improved Model

Configuration: 3-layer encoder with BatchNorm and Dropout(0.3), 20 epochs, $lr=1e-4$.

Result: $IoU = 0.4547$, $F1 = 0.6231$. The model identifies salient regions with much higher accuracy and clearer object boundaries.

Analysis: Three improvements work together: deeper encoder increases representational power, BatchNorm stabilizes training, and Dropout improves generalization. 20 epochs allow full convergence.

6.3 Experiment Conclusion

The improved model significantly outperforms the baseline across all metrics. A 57.8% increase in IoU demonstrates the strong impact of architectural and training improvements. This highlights the importance of BatchNorm, Dropout, and model depth in segmentation tasks.

7. Visualization and Demo

7.1 Prediction Visualizations

Predictions were visualized in four panels:

- Input Image
- Ground Truth
- Predicted Mask (threshold = 0.4)
- Overlay (input image + predicted saliency mask, red channel, alpha 0.5)

Visualizations confirm that the improved model successfully identifies dominant objects in most test images, with clearly separated salient regions.

7.2 Live Inference Demo

A full interactive demo was implemented in the Google Colab notebook. The user can upload an image and immediately receive a saliency prediction. Steps:

- Upload image via `files.upload()`
- Preprocess (BGR→RGB, resize 128×128, normalization)
- Convert to PyTorch tensor and send to device
- Model performs inference and `torch.sigmoid()` is applied
- Binary prediction generated using threshold 0.4
- Three panels displayed: Input / Saliency Mask / Overlay
- Inference time displayed in milliseconds

Average inference time is under 50ms on CPU and under 10ms on GPU, making the system suitable for real-time applications.

8. Code Structure and Deliverables

8.1 Project Files

File	Description
<code>data_loader.py</code>	Data loading, preprocessing, augmentation
<code>sod_model.py</code>	CNN architecture (BaselineSOD and ImprovedSOD)
<code>train.py</code>	Training and validation loop with logging
<code>evaluate.py</code>	IoU, Precision, Recall, F1 metrics
<code>demo_notebook.ipynb</code>	Full interactive demo notebook
<code>sod_model.pth</code>	Saved trained model

8.2 Tools and Technologies

- Language: Python 3.9+
- Framework: PyTorch
- Libraries: NumPy, OpenCV, Matplotlib, scikit-learn, tqdm
- Environment: Google Colab with GPU
- Version Control: Git / GitHub

9. Lessons and Conclusions

9.1 Key Learnings

This project provided deep practical understanding in:

- Building a full ML pipeline from scratch
- Encoder-decoder architecture design
- Spatial information loss in MaxPooling and reconstruction via ConvTranspose
- Importance of regularization techniques
- Segmentation evaluation using IoU
- Visualization as a debugging tool

9.2 Limitations

- No skip connections (like U-Net), which would preserve spatial details
- Input size 128×128 loses fine-grained image details
- No data augmentation reduces generalization ability

9.3 Future Work

- Implement skip connections (U-Net architecture)
- Increase input resolution to 224×224
- Add data augmentation for better generalization
- Experiment with combined loss functions (BCE + Dice Loss)
- Test on other datasets like ECSSD and MSRA10K

9.4 Final Conclusion

This project successfully implemented a complete Salient Object Detection system from scratch, demonstrating deep understanding of the machine learning pipeline. The improved model achieved $\text{IoU} = 0.4547$ and $\text{F1} = 0.6231$, with a significant improvement of over 57% compared to the baseline. The interactive demo shows that the system works efficiently on unseen images. The project fulfills all technical requirements and demonstrates readiness for practical computer vision applications.